

Software de automatización de administración y control de gastos
comunes

(DAS) Documento Arquitectura de Software
Versión 1.0

Integrantes: Bastián Benjamín González Flores

Profesor: Ricardo Pino

Asignatura: Ingeniería de Software

Identificación de Documento

Identificación	001
Proyecto	Software de automatización de administración y control de gastos comunes
Versión	1.0.1
Documento mantenido por	Bastian González Flores
Fecha de última revisión	—
Fecha de próxima revisión	
Documento aprobado por	Ricardo Pino
Fecha de última aprobación	—

Historia de Revisiones

Fecha	Versión	Descripción	Autor
03 - 04 - 2026	1.0.0	Implementación de introducción desde el punto 1.1 hasta 1.6	Bastian Gonzalez
04 - 04 - 2026	1.0.1	Implementación de la vision del sistema desde el punto 2.1 hasta el 2.5	Bastian Gonzalez
05 - 04 - 2026	1.0.2	Refinamiento de RF y RNF	Bastián González

Tabla de Contenidos

1. INTRODUCCIÓN.....	4
1.1. CONTEXTO DEL PROBLEMA.....	4
1.2. PROPÓSITO.....	4
1.3. ÁMBITO.....	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES.....	4
1.5. RESUMEN EJECUTIVO.....	4
1.6. ARQUITECTURA DEL SISTEMA.....	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA.....	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS.....	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS.....	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS.....	5
4.1. VISTA DE ESCENARIO.....	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA.....	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS.....	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA.....	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD.....	7

5.1. PROPÓSITO.....	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO.....	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO.....	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS.....	8
9. CONTROL DE VERSIONES.....	8
9.1. PROPÓSITO.....	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL).....	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES.....	8
8. BIBLIOGRAFÍA.....	8
9. ANEXOS	8
9.1. CARTA GANTT.....	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

El edificio *El mirador*, ubicado en la zona centro de la ciudad, enfrenta una serie de dificultades en la gestión de sus gastos comunes y a su vez, en la administración de los servicios asociados. La presente alta densidad de residentes (160 departamentos distribuidos en 20 pisos, con un 60% de propietarios y un 40% de arrendatarios) genera una dinámica comunitaria compleja, con intereses diversos y necesidades diferenciadas.

Actualmente, la administración del edificio está fuertemente limitada por procesos manuales y poco automatizados, lo que causa ineficiencias en la gestión de cobros, retrasos en la identificación y manejo de morosidades y una comunicación precaria con los residentes. Estas carencias afectan directamente en el flujo de la caja del edificio, dificultando en demasía el pago de servicios esenciales como luz, agua, gas, mantención de ascensores y remuneraciones del personal.

A esto se le suma la insuficiencia de recursos humanos (Solo tres personas de mantención y dos conserjes) la cual empeora la situación, debido que la capacidad de respuesta ante demandas extraordinarias y imprevistos resulta deficiente. A su vez, la falta de transparencia en la administración genera desconfianza entre los mismos residentes, afectando la cohesión comunitaria y también, afectando de manera negativa la toma de decisiones colectivas.

Finalmente y, como consecuencia, el edificio enfrenta riesgos financieros, operativos y sociales potencialmente críticos.

1.2. Propósito

El propósito principal de este proyecto es diseñar e implementar un sistema de software que automatice la gestión financiera del edificio y *El Mirador*. Esta solución busca aumentar la eficiencia de los procesos administrativos y reducción de morosidad a través de procesos automatizados que permitan un flujo de caja estable, garantizando que el equipo de administración pueda cumplir correctamente con sus obligaciones operativas y financieras del edificio.

El sistema se plantea como una solución tecnológica que proporcione confianza y transparencia a los residentes, otorgándoles acceso en tiempo real a la información sobre pagos, morosidades y solicitudes. A su vez, mejorará la comunicación entre la comunidad y administración, mejorando y fortaleciendo la colaboración y también reduciendo los conflictos ocurridos por causa de la falta de información o de procesos ineficientes.

Bajo términos generales, el propósito es aportar para una administración más eficiente, participativa y confiable, que garantice la sostenibilidad financiera del edificio y el bienestar de la comunidad de residentes a través del uso de herramientas tecnológicas modernizadas, automatizadas y completamente accesible. El desarrollo del sistema se llevará a cabo bajo un modelo de cascada, permitiendo un proceso estructurado y

controlado en cada etapa.

1.3. Ámbito

El proyecto presente se basa en el desarrollo de un sistema software orientado a la automatización de la administración general y control de los gastos comunes del edificio *El Mirador*. Su alcance incluye tanto las especificaciones destinadas al equipo de administración como también las dirigidas a los residentes, ofreciendo una interacción eficiente y transparente para ambas partes.

El sistema cubrirá los siguientes aspectos principales:

1.3.1 Gestión de residentes: Actualización, registro y clasificación de propietarios y arrendatarios.

1.3.2 Gestión de pagos: Cálculo de cuotas según características de los departamentos, registro de pagos, emisión de recibos y control de morosidades.

1.3.3 Solicitudes y quejas: Recepción, seguimiento y resolución de solicitudes de mantenimiento y reclamos de residentes.

1.3.4 Generación de consultas e informes: Creación de informes financieros, de morosidad y de gestión administrativa y que estos mismos sean accesibles en tiempo real.

El ámbito del proyecto se limita a la mejora de la administración interna del edificio. El sistema se integrará con plataformas de pago externas (como WebPay) para delegar el procesamiento de transacciones, limitándose a registrar estados de pago, recibir confirmaciones y mantener trazabilidad financiera. Está en consideración la capacidad de escalabilidad futura, que abra paso a la extensión del sistema a otros edificios u otras comunidades con características similares a las del edificio *El Mirado*. Y a su vez, el alcance definido será implementado siguiendo el modelo de desarrollo en cascada, asegurando que cada fase (análisis, diseño, implementación, pruebas y despliegue) se complete de manera secuencial y estrictamente documentada.

1.4. Definiciones, acrónimos y abreviaciones

ACRONIMO	DESCRIPCION
<i>RF</i>	Requerimiento Funcional: Especifica una funcionalidad que el sistema debe implementar.
<i>RNF</i>	Requerimiento No Funcional: Especifica una característica de calidad o restricción del sistema.
<i>Administrador</i>	Usuario responsable de la gestión general del edificio, incluyendo pagos, residentes y proveedores.
<i>Residente</i>	Persona que habita un departamento del edificio, ya sea propietario o arrendatario.
<i>Morosidad</i>	Estado de atraso en el pago de cuotas de gastos comunes.

<i>Gastos comunes</i>	Conjunto de costos compartidos por los residentes (luz, agua, gas, mantención, remuneraciones, etc.)
<i>Prorratio</i>	Método de cálculo proporcional que distribuye gastos comunes según características del departamento (m2, piso, número de piezas, etc.)
<i>Informe de gestión</i>	Reporte generado por el sistema que resume información administrativa, financiera y operativa.
<i>WCAG nivel AA</i>	Estándar internacional de accesibilidad web que asegura que el sistema sea usable por personas con discapacidad, cumpliendo criterios de contraste, navegación y comprensión.
<i>W3C</i>	(World Wide Web Consortium) Organización internacional que define estándares y buenas prácticas para el desarrollo de aplicaciones y servicios web.
<i>Modelo en Cascada</i>	Metodología de desarrollo de software tradicional que organiza el proyecto en fases secuenciales (análisis, diseño, implementación, pruebas y despliegue), donde cada etapa debe completarse antes de iniciar la siguiente.
<i>SMTP</i>	Simple Mail Transfer Protocol (Protocolo Simple de Transferencia de Correo). Protocolo estándar utilizado para el envío de correos electrónicos desde el sistema hacia los usuarios (residentes y administrador).

1.5. Resumen ejecutivo

El presente proyecto de desarrollo del sistema de automatización para la administración de ámbitos financieros y sociales del edificio *El Mirador* surge como una solución a los desafíos y carencias actuales de gestión financiera y operativa que enfrenta la comunidad. Esta iniciativa busca transformar procesos manuales e ineficientes en un modelo digital, confiable y transparente, que permita mejorar la calidad de vida de los residentes y fortalecer la sostenibilidad del edificio.

La propuesta del proyecto se basa en cuatro pilares fundamentales:

1.5.1 Optimización administrativa: Reducción de tiempos, errores y complicaciones mediante la automatización de registros, cálculos y reportes.

1.5.2 Eficiencia financiera: Se propone una disminución de la morosidad y a su vez un fortalecimiento del flujo de caja para asegurar el cumplimiento correcto de las obligaciones.

1.5.3 Atención a residentes: Implementación de canales formales para solicitudes y reclamos, con seguimiento claro y resolutivo para mejorar la comunicación y porcentaje de satisfacción entre los residentes y el equipo de administración.

1.5.4 Transparencia en la gestión: Acceso en tiempo real a información relevante, priorizando el fortalecimiento de la confianza y la participación comunitaria.

En síntesis, el sistema software representa una oportunidad de modernización y continua mejora, alineando la gestión del edificio con estándares tecnológicos actuales, asegurando un impacto positivo tanto en la administración como en los

residentes y la ejecución se realizará bajo un modelo de cascada, lo que permitirá mantener trazabilidad, control y calidad en cada fase del desarrollo.

1.6. Arquitectura del sistema

La arquitectura propuesta para el sistema de administración financiero del edificio *El Mirador* se fundamenta en un modelo de desarrollo en cascada, dado que el proyecto se orienta a un enfoque tradicional de ingeniería software.

Principales vistas y componentes a considerar son:

1.6.1 Vista de Escenario: Identificación de actores principales (Administrador, residentes) y sus respectivas interacciones con el sistema mediante casos de uso como registro de pagos, gestión de morosidades y solicitudes de mantención.

1.6.2 Vista Lógica: Organización del sistema en capas bien definidas

- Capa de presentación (UI): interfaz web y móvil tanto para residentes como administración.
- Capa de negocio (Lógica del sistema): Lógica de cálculo de cuotas, gestión de pagos, solicitudes y generación de informes.
- Capa de datos (Persistencia): Almacenamiento seguro de información de residentes, registros administrativos y transacciones.

1.6.3 Vista de Desarrollo/implementación: Definición de componentes de software y empaquetamiento modular, con posibilidades de futuras integraciones externas.

1.6.3 Vista de Procesos: Representación de los flujos internos, como cálculo automático de cuotas, generación de informe y gestión de solicitudes.

1.6.4 Vista Física: Infraestructura de despliegue, incluyendo servidores de aplicación, base de datos y acceso desde dispositivos de los residentes.

El beneficio del modelo en cascada es que éste permite que cada una de estas vistas se construyan de manera secuencial, garantizando que los requerimientos iniciales se conviertan en un diseño sólido, que luego se implemente, pruebe y despliegue con un control estricto y completo de calidad.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema de administración de gastos comunes del edificio *El Mirador* nace como una plataforma digital integral que garantizará la automatización de procesos administrativos, financieros y operativos de la comunidad. El diseño presentado responde a la necesidad de superar las limitaciones actuales de gestión manual, otorgando una solución eficiente, confiable y transparente para todos los involucrados.

El sistema estará compuesto por una interfaz (UI) accesible e intuitiva tanto para el equipo de administración como para los residentes, permitiendo una interacción ordenada y clara entre ambos. Dentro de la plataforma se podrán realizar las siguientes acciones principales: Administración de residentes, Registro y seguimiento financiero (con integración a plataformas de pago externas), Atención de solicitudes y quejas, y Generación de consultas e informes.

En términos generales, el sistema busca reducir la morosidad, mejorar el flujo de la caja del edificio, optimizar la eficiencia administrativa mediante la automatización de procesos, crear y fortalecer la confianza y participación de los residentes y sentar las bases para una futura escalabilidad hacia otros edificios o comunidades.

2.2. Objetivos del sistema

El sistema de administración de gastos comunes del edificio El Mirador busca cumplir una serie de objetivos estratégicos y operativos que garanticen su utilidad, eficiencia y sostenibilidad a lo largo del tiempo. Estos objetivos se clasifican en generales y específicos:

2.2.1 Objetivo General

Desarrollar un sistema de software que automatice la gestión de los gastos comunes del edificio El Mirador, optimizando los procesos financieros y administrativos, disminuyendo la morosidad y reforzando la transparencia y la confianza entre la administración y los residentes.

2.2.2 Objetivos Específicos

- Optimizar la gestión de residentes: permitir el registro, actualización y clasificación de propietarios y arrendatarios de forma ordenada y de fácil acceso.
- Gestionar el ciclo financiero: calcular las cuotas mediante prorratio, registrar los estados de pago a partir de confirmaciones externas (WebPay u otras pasarelas),

generar comprobantes y realizar un control efectivo de las morosidades.

- Mejorar la atención de solicitudes y reclamos: implementar un sistema eficiente para la recepción, seguimiento y resolución de quejas y solicitudes de mantenimiento.
- Facilitar la generación de informes: producir reportes financieros, administrativos y de morosidad en tiempo real, disponibles tanto para la administración como para los residentes.
- Garantizar estándares de calidad: asegurar la accesibilidad (WCAG nivel AA), el cumplimiento de buenas prácticas (W3C) y la seguridad en el tratamiento de los datos.
- Asegurar escalabilidad futura: diseñar el sistema con la capacidad de ampliarse a otros edificios o comunidades con características similares.
- Mantener trazabilidad y control: aplicar el modelo de desarrollo en cascada para garantizar un proceso estructurado, debidamente documentado y verificable en cada fase.

2.3. Requerimientos funcionales

A continuación se detallan los requerimientos funcionales del sistema,

organizados por módulos principales:

2.3.1 Gestión de usuarios, roles y acceso.

ID	Requerimiento	Descripción
RF-01	Gestión de usuarios	El sistema debe permitir el registro, actualización y desactivación de usuarios, incluyendo información de identificación, contacto, rol, medio de notificación y estado. No se contempla eliminación física.
RF-02	Gestión de roles y permisos	El sistema debe permitir la asignación y actualización de roles y permisos diferenciados según tipo de usuario.
RF-03	Gestión de perfiles de usuario	El sistema debe permitir el registro, actualización y desactivación de perfiles de usuario, asegurando control de acceso.

2.3.2 Gestión de residentes y unidades.

ID	Requerimiento	Descripción
RF-04	Gestión de residentes	El sistema debe permitir el registro, actualización y desactivación de residentes, manteniendo trazabilidad histórica.
RF-05	Gestión de unidades habitacionales	El sistema debe permitir el registro, actualización y desactivación de departamentos.

2.3.3 Gestión financiera y cobranza.

ID	Requerimiento	Descripción
RF-06	Cálculo automático de cuotas	El sistema debe permitir el registro y actualización de parámetros de cálculo y la generación automática de cuotas.
RF-07	Gestión de gastos comunes y pagos	El sistema debe permitir el registro, actualización y desactivación (anulación lógica) de pagos de gastos comunes. El registro se realiza a partir de confirmaciones recibidas desde el Sistema de Pagos Externo.
RF-08	Gestión de gastos extraordinarios	El sistema debe permitir el registro, actualización y desactivación de gastos extraordinarios.
RF-09	Generación de recibos de pago	El sistema debe permitir la generación y reimpresión de comprobantes de pago a partir de transacciones confirmadas por el Sistema de Pagos Externo.
RF-10	Gestión de morosidad	El sistema debe permitir el registro automático, actualización y desactivación automática del estado de morosidad, basándose en el estado de los pagos registrados.

2.3.4 Gestión de notificaciones y comunicaciones.

ID	Requerimiento	Descripción
RF-11	Configuración de notificaciones	El sistema debe permitir el registro, actualización y activación/desactivación de notificaciones automáticas.
RF-12	Programación de notificaciones	El sistema debe permitir el registro, actualización y activación/desactivación de notificaciones asociadas a eventos.
RF-13	Notificaciones múltiples	El sistema debe permitir el envío de notificaciones a múltiples usuarios relacionados con una propiedad.
RF-14	Historial de comunicaciones	El sistema debe permitir el registro automático, consulta y conservación del historial de notificaciones enviadas.

2.3.5 Gestión de solicitudes y atención.

ID	Requerimiento	Descripción
RF-15	Registro de solicitudes de mantenimiento	El sistema debe permitir el registro, actualización y cierre (desactivación) de solicitudes.
RF-16	Gestión de solicitudes de quejas	El sistema debe permitir el registro, actualización y cierre (desactivación) de reclamos.

2.3.6 Consultas, visualización y control.

ID	Requerimiento	Descripción
RF-17	Acceso a información en tiempo real	El sistema debe permitir la consulta en línea de información relevante según perfil de usuario.
RF-18	Consulta de información por parte de residentes	El sistema debe permitir la visualización de estado de cuenta, pagos y deudas.
RF-19	Consulta administrativa avanzada	El sistema debe permitir la configuración y actualización de criterios de búsqueda para consultas dinámicas.
RF-20	Búsqueda y filtros	El sistema debe permitir la configuración y uso de filtros avanzados de búsqueda.
RF-21	Panel de control administrativo	El sistema debe permitir la visualización de indicadores clave de gestión.

2.3.7 Reportes y análisis.

ID	Requerimiento	Descripción
RF-22	Generación de informes generales	El sistema debe permitir la generación de informes con configuración y actualización de parámetros.
RF-23	Generación de reportes financieros	El sistema debe permitir la generación y configuración de reportes financieros.
RF-24	Reporte de morosidad	El sistema debe permitir la generación de reportes con actualización de filtros.
RF-25	Reporte de pagos realizados	El sistema debe permitir la generación de reportes con criterios configurables.
RF-26	Reporte de gastos comunes y extraordinarios	El sistema debe permitir la generación y configuración de reportes.
RF-27	Reporte de estado de cuenta por residente	El sistema debe permitir la generación de estados de cuenta individuales.
RF-28	Reporte de solicitudes y mantenimiento	El sistema debe permitir la generación de reportes sobre solicitudes.
RF-29	Exportación de reportes	El sistema debe permitir la generación y exportación de reportes en distintos formatos.
RF-30	Programación de reportes automáticos	El sistema debe permitir el registro, actualización y activación/desactivación de reportes periódicos.

2.4. Requerimientos no funcionales

ID	Requerimiento	Descripción
RNF-01	Usabilidad	El sistema debe ser intuitivo, fácil de usar y comprensible para usuarios administrativos y residentes con distintos niveles de alfabetización digital.
RNF-02	Accesibilidad	La interfaz debe cumplir criterios básicos de accesibilidad visual y de navegación, facilitando su uso por personas con distintas capacidades.
RNF-03	Compatibilidad	El sistema debe ser accesible desde navegadores web modernos y dispositivos móviles.
RNF-04	Rendimiento	El sistema debe responder las operaciones comunes de consulta en un tiempo no mayor a 3 segundos bajo carga normal.
RNF-05	Soporte multiusuario	El sistema debe permitir el uso simultáneo de múltiples usuarios sin degradación crítica del servicio.
RNF-06	Escalabilidad	El sistema debe adaptarse al crecimiento en número de usuarios, propiedades y transacciones sin afectar su desempeño.
RNF-07	Disponibilidad	El sistema debe estar disponible al menos el 99% del tiempo mensual, excluyendo mantenimientos programados.
RNF-08	Confiabilidad	El sistema debe evitar pérdida de información y asegurar consistencia en los datos transaccionales.
RNF-09	Respaldo y recuperación	El sistema debe realizar respaldos automáticos y permitir la recuperación ante fallas o errores.
RNF-10	Seguridad de acceso	El sistema debe implementar autenticación segura y control de acceso basado en roles y permisos.
RNF-11	Protección de datos	El sistema debe garantizar la confidencialidad e integridad de los datos personales, financieros y operativos.
RNF-12	Trazabilidad	El sistema debe mantener registros auditables de operaciones críticas como pagos, cobros, cambios y notificaciones.
RNF-13	Calidad de la información	El sistema debe validar datos obligatorios, formatos, duplicidades e inconsistencias antes de almacenarlos.
RNF-14	Notificaciones confiables	El sistema debe asegurar el registro del envío de notificaciones y gestionar reintentos en caso de fallas.
RNF-15	Mantenibilidad	El sistema debe estar diseñado de forma modular para facilitar mantenimiento, correcciones y mejoras.
RNF-16	Configurabilidad	El sistema debe permitir parametrizar reglas de negocio sin necesidad de modificar el código fuente.
RNF-17	Localización y contexto normativo	El sistema debe permitir su adaptación configurable a distintos contextos, incluyendo idioma, moneda, formato de fecha y reglas de cobro, sin requerir cambios en el código fuente.
RNF-18	Interoperabilidad	El sistema debe permitir la integración con sistemas externos como plataformas de pago (e.g., WebPay), correo electrónico y mensajería, a través de interfaces o APIs definidas, sin asumir responsabilidad directa sobre el procesamiento de transacciones.

2.5. Supuestos y dependencias

A continuación se detallan los supuestos y dependencias identificados para el desarrollo y correcto funcionamiento del sistema software de automatización de administración y control de gastos comunes del edificio *El Mirador*.

2.5.1 Supuestos:

Los siguientes supuestos se consideran válidos durante todo el ciclo de vida del proyecto:

- Se cuenta con la disponibilidad y colaboración activa del administrador del edificio y de al menos un representante de los residentes para la validación de

requerimientos, revisión de prototipos y pruebas de aceptación del sistema.

- Los residentes y el personal del edificio (conserjes y mantención) poseen un nivel básico de alfabetización digital y acceso a dispositivos con conexión a internet (computadores, tablets o smartphones) para utilizar el sistema.
- La información inicial de residentes, departamentos, personal y gastos comunes será proporcionada por la administración del edificio de manera completa, precisa y en formato digital (o fácilmente digitalizable).
- El edificio mantiene una estructura organizacional estable, con un administrador claramente definido como responsable principal de la gestión del sistema.
- No se producirán cambios significativos en el número de departamentos (160) ni en la distribución de propietarios y arrendatarios durante la fase de desarrollo del proyecto.
- El modelo de desarrollo en cascada será seguido estrictamente, lo que implica que los requerimientos funcionales y no funcionales permanecerán estables una vez aprobada esta etapa de visión del sistema.
- El sistema se integrará con pasarelas de pago externas como WebPay, delegando en ellas el procesamiento de transacciones. El sistema solo registrará el estado de la transacción a partir de la confirmación recibida.

2.5.2 Dependencias:

El éxito del proyecto depende de los siguientes factores externos e internos:

- Dependencia tecnológica: El sistema requiere de una infraestructura de hosting web confiable y una base de datos relacional para almacenar y gestionar la información de residentes, pagos y solicitudes.
- Dependencia de recursos humanos: Se necesita la participación continua del administrador del edificio para la carga inicial de datos, definición de reglas de prorrateo y validación de flujos de negocio.
- Dependencia de notificaciones: El envío de notificaciones automáticas (recordatorios de pago, confirmaciones y alertas) depende de la disponibilidad de un servicio de correo electrónico externo confiable (SMTP) o de una pasarela de mensajería integrada.
- Dependencia de acceso: Los residentes y el administrador requieren conexión a internet estable para acceder al sistema desde cualquier dispositivo.
- Dependencia de estándares: El cumplimiento de accesibilidad (WCAG nivel AA) y buenas prácticas web (W3C) depende de la correcta implementación de

las tecnologías front-end seleccionadas.

- Dependencia externa: El sistema se integra con plataformas de pago externas (como WebPay) a través de APIs definidas. El procesamiento de transacciones es responsabilidad exclusiva de dichas plataformas; el sistema solo registra el estado de pago a partir de la confirmación recibida.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado:

El sistema adopta el estilo de Arquitectura en Capas (Layered Architecture / N-Tier), organizado en tres capas bien diferenciadas:

Capa de Presentación (UI): Interfaz web y móvil accesible para el Administrador del Sistema, Residentes, Personal del Edificio y Comité de Administración. Concentra la lógica de visualización e interacción con el usuario, sin incorporar reglas de negocio.

Capa de Lógica de Negocio (Servicios): Gestiona el procesamiento de reglas administrativas, cálculo de cuotas, gestión de morosidades, envío de notificaciones e integración con sistemas externos como el Sistema de Pagos Externo (WebPay) y el Servicio de Notificaciones. Actúa como intermediaria entre la presentación y los datos.

Capa de Datos (Persistencia): Gestiona el almacenamiento, consulta y actualización de la información en una base de datos relacional. Incluye la trazabilidad de operaciones críticas exigida por los RNF de seguridad y auditoría.

3.2. Justificación del estilo según el contexto del sistema:

La elección de la Arquitectura en Capas responde directamente al contexto del sistema por las siguientes razones:

Coherencia con el modelo de desarrollo: El proyecto utiliza un modelo en cascada, en el que cada fase se documenta y valida antes de avanzar. La arquitectura en capas es la más adecuada para este enfoque, ya que sus límites bien definidos facilitan la documentación progresiva y el control de calidad en cada etapa.

Separación de responsabilidades: Los módulos del sistema (gestión de residentes, financiero, notificaciones, reportes) tienen responsabilidades claramente delimitadas que se mapean de forma natural a las tres capas,

reduciendo el acoplamiento entre componentes.

Integración con sistemas externos: La Capa de Lógica de Negocio actúa como punto de control para las integraciones con el Sistema de Pagos Externo y el Servicio de Notificaciones, aislando al resto del sistema de cambios en dichas plataformas externas.

Escalabilidad y mantenibilidad: El diseño en capas permite modificar o reemplazar componentes de una capa sin afectar las demás, facilitando la escalabilidad futura hacia otros edificios o comunidades (ref RNF-06) y la mantenibilidad del sistema (ref RNF-15).

Consistencia con la documentación existente: La Vista Lógica descrita en la sección 1.6 del presente documento ya describe explícitamente tres capas (presentación, negocio y datos), por lo que el estilo adoptado es congruente con la arquitectura definida desde la etapa de planificación.

3.3 Patrones de diseño aplicados:

Los siguientes patrones de diseño son aplicados al sistema, complementando el estilo arquitectónico en capas:

MVC (Model-View-Controller): Aplicado en la Capa de Presentación. Separa la interfaz de usuario (View), la lógica de control de flujo (Controller) y los datos del dominio (Model), facilitando el mantenimiento de la UI sin impactar la lógica de negocio.

Repositorio (Repository Pattern): Aplicado en la Capa de Datos. Abstrae el acceso a la base de datos, exponiendo una interfaz uniforme a la Capa de Lógica de Negocio. Esto permite cambiar el motor de base de datos sin modificar la lógica del sistema, favoreciendo RNF-15 (Mantenibilidad) y RNF-16 (Configurabilidad).

Observador (Observer Pattern): Aplicado en el módulo de notificaciones y morosidad. Cuando el sistema detecta un cambio de estado relevante (ej. cuota vencida, cambio a estado moroso), notifica automáticamente a los componentes suscritos (Servicio de Notificaciones, registro de auditoría), alineándose con RF-10, RF-11 y RF-12.

Fachada (Facade Pattern): Aplicado en los puntos de integración con sistemas externos. Se define una interfaz simplificada hacia el Sistema de Pagos Externo y el Servicio de Notificaciones, ocultando la complejidad de las APIs externas al resto del sistema y encapsulando los puntos de cambio ante actualizaciones de

dichas plataformas (ref RNF-18).

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

1.1. VISTA DE ESCENARIO

1.1.1. Propósito

La Vista de Escenario, también denominada Vista de Casos de Uso, representa los requisitos funcionales del sistema desde la perspectiva de los actores que interactúan con él. Su propósito es describir el comportamiento esperado del sistema en términos de acciones concretas, siendo un hilo conductor que valida y articula las demás vistas arquitectónicas. Para el Sistema El Mirador, esta vista centraliza los flujos de gestión administrativa, financiera y de reportes que el sistema debe soportar.

1.1.2. Actores

Administrador del Sistema (actor interno principal): Responsable de la gestión completa del edificio. Interactúa con la totalidad de los módulos del sistema: gestión de usuarios y unidades, control financiero, morosidad, notificaciones y generación de reportes.

Residente — Propietario o Arrendatario (actor interno): Usuario que consulta su estado de cuenta, realiza pagos a través de plataformas externas, ingresa solicitudes de mantenimiento y recibe notificaciones del sistema.

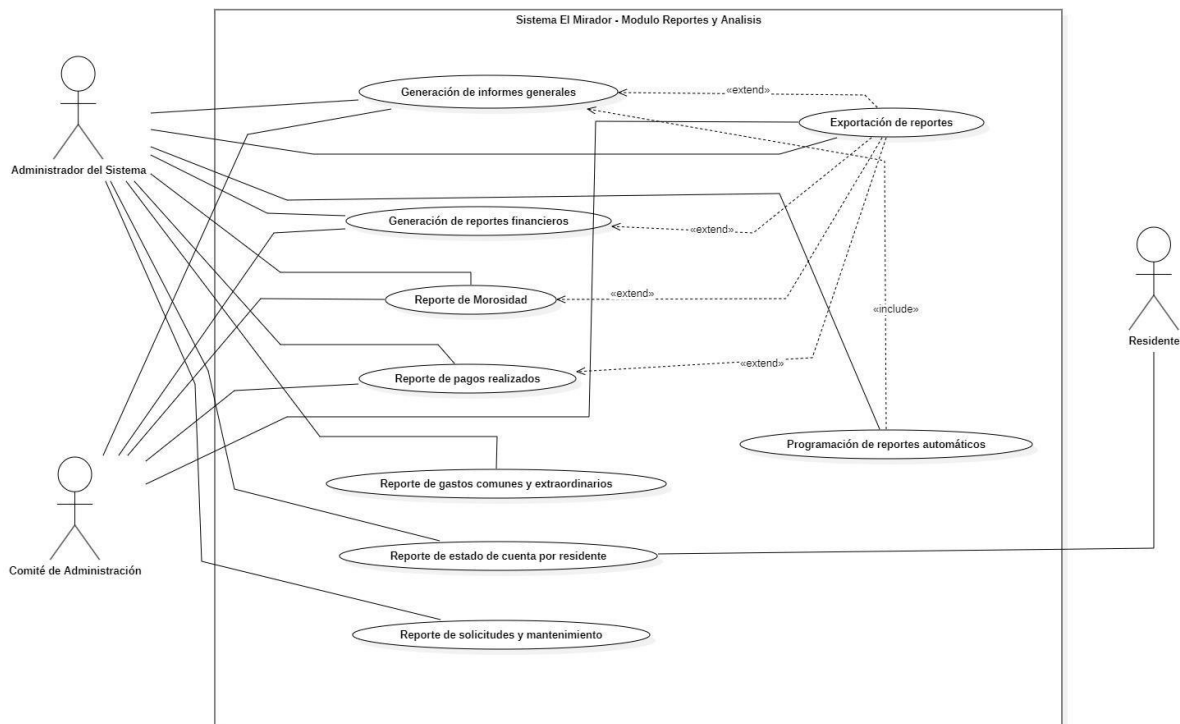
Comité de Administración (actor interno secundario): Grupo de residentes con acceso a reportes financieros y operativos para supervisión y toma de decisiones. No opera el sistema directamente.

Personal del Edificio (actor interno secundario): Conserjes y personal de mantención. Consultan y actualizan el estado de solicitudes de mantenimiento asignadas.

Sistema de Pagos Externo — WebPay (actor externo): Plataforma externa que procesa las transacciones de pago de manera autónoma y retorna el estado de cada transacción al sistema para su registro.

Servicio de Notificaciones (actor externo): Servicio externo responsable del envío de correos electrónicos, SMS u otros mensajes a los residentes según las reglas configuradas por el administrador.

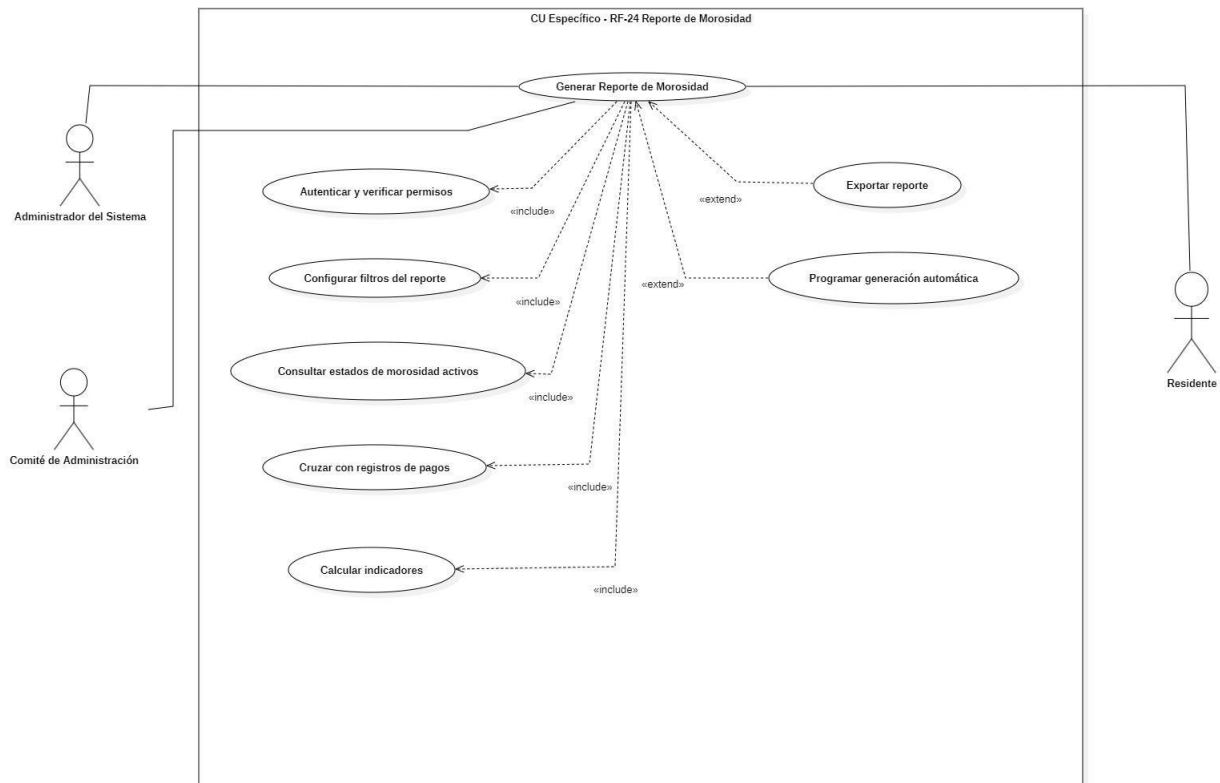
1.1.3. Diagrama general de casos de uso



El diagrama general muestra el módulo de Reportes y Análisis, encargado de reunir los casos de uso más relevantes para la estrategia del sistema. Se distinguen nueve casos de uso principales (RF-22 a RF-30), tres actores involucrados y las conexiones de extensión e inclusión que estructuran el módulo. El Administrador del Sistema es el principal responsable de iniciar la mayoría de los procesos; el Comité de Administración interviene como actor secundario en los reportes financieros y de morosidad; el Residente solo puede consultar su propio estado de cuenta (RF-27). Las relaciones «extend» entre RF-29 (Exportar reporte) y RF-22, RF-23, RF-24, y RF-25 muestran que la exportación es un proceso opcional que amplía los flujos de generación. Por otro lado, la relación «include» de RF-30 (Reportes automáticos) a RF-22 implica que la programación de reportes

siempre incorpora la lógica general de generación.

1.1.4. Diagrama de casos de uso específicos



El diagrama específico describe todo el proceso del caso de uso RF-24. El punto central es la función 'Generar Reporte de Morosidad', a la que pueden acceder el Administrador del Sistema, el Comité de Administración y el Residente, cada uno con distintos niveles de acceso. El flujo contempla cinco pasos obligatorios: autenticación y validación de permisos (RF-01/RF-03), selección de filtros para el reporte, consulta de morosidades vigentes (RF-10), comparación con los registros de pagos (RF-07/RF-25) y el cálculo de indicadores. Además, existen dos flujos opcionales: la exportación del reporte (RF-29) y la programación para su generación automática (RF-30).

1.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-01	Generar Reporte de Morosidad	Administrador, Comité, Residente
CU-02	Generar Reporte Financiero	Administrador, Comité
CU-03	Exportar Reporte	Administrador, Comité
CU-04	Registrar Estado de Morosidad	Sistema (automático)

1.1.6. Especificación de casos de uso

CU-01 – Generar Reporte de Morosidad

Caso de Uso	Generar Reporte de Morosidad	Identificador: CU-01
Actores	Administrador del Sistema (principal), Comité de Administración, Residente	
Tipo	Primario	
Referencias	RF-24, RF-10, RF-07, RF-25, RF-29, RF-30. Relacionados: CU-03, CU-04.	
Precondición	El actor debe estar autenticado con los permisos correspondientes. Debe existir al menos un período de gastos comunes registrado.	
Postcondición	El sistema genera y muestra el reporte de morosidad actualizado. El reporte queda disponible para exportación o programación automática.	
Descripción	Permite generar un reporte consolidado del estado de morosidad del edificio, configurando filtros por período, unidad y estado.	
Resumen	El actor accede al módulo de Reportes, configura filtros, y el sistema consulta estados de morosidad, cruza con registros de pagos, calcula indicadores y presenta el reporte.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Administrador	Accede al módulo de Reportes y selecciona Reporte de Morosidad.
2	Sistema	Verifica autenticación y permisos del usuario sobre el módulo de reportes.
3	Sistema	Presenta formulario de configuración de filtros.
4	Administrador	Configura filtros: período, unidad(es), estado de morosidad (activo/histórico).
5	Sistema	Consulta estados de morosidad activos/históricos según filtros aplicados.
6	Sistema	Cruza resultados con registros de pagos confirmados por el Sistema de Pagos Externo (RF-07).
7	Sistema	Calcula indicadores: total de unidades morosas, monto total de deuda, distribución por período
8	Sistema	Genera y presenta el reporte de morosidad al actor.

9	Administrador	Visualiza el reporte. Fin del caso de uso.
---	---------------	--

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
4a	Si el actor no selecciona filtros, el sistema aplica el período vigente por defecto y continúa en paso 5
8a	El actor selecciona Exportar reporte. Se extiende CU-03 y al finalizar retorna al paso 9.
8a	El actor selecciona Programar reporte automático. Se extiende RF-30 y al finalizar retorna al paso 9.
2 – Exc.	Sin permisos: el sistema muestra mensaje de acceso denegado y termina el caso de uso.
5 – Exc.	Sin registros en el período: el sistema muestra reporte vacío con indicadores en cero.

CU-02 – Generar Reporte Financiero

Caso de Uso	Generar Reporte Financiero	Identificador: CU-02
Actores	Administrador del Sistema (principal), Comité de Administración, Residente	
Tipo	Primario	
Referencias	RF-24, RF-10, RF-07, RF-25, RF-29, RF-30. Relacionados: CU-03, CU-04.	
Precondición	El actor debe estar autenticado con los permisos correspondientes. Debe existir al menos un período de gastos comunes registrado.	
Postcondición	El sistema genera y muestra el reporte de morosidad actualizado. El reporte queda disponible para exportación o programación automática.	
Descripción	Permite generar un reporte consolidado del estado de morosidad del edificio, configurando filtros por período, unidad y estado.	
Resumen	El actor accede al módulo de Reportes, configura filtros, y el sistema consulta estados de morosidad, cruza con registros de pagos, calcula indicadores y presenta el reporte.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Administrador	Accede al módulo de Reportes y selecciona Reporte Financiero.
2	Sistema	Verifica autenticación y permisos del usuario.
3	Sistema	Presenta formulario de configuración de parámetros.
4	Administrador	Configura parámetros: período, tipo de gasto (común/extraordinario/ambos), rango de fechas.
5	Sistema	Consulta los pagos registrados en el período a partir de confirmaciones del Sistema de Pagos Externo.
6	Sistema	Consolida montos por categoría: total recaudado, total pendiente, gastos extraordinarios.
7	Sistema	Genera y muestra el reporte financiero al actor.
8	Administrador	Visualiza el reporte. Fin del caso de uso.

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
4a	Sin selección de tipo de gasto: el sistema incluye todos los tipos para el período.
7a	El actor selecciona Exportar. Se extiende CU-03 y al finalizar retorna al paso 8.
8a	Sin permisos: el sistema muestra acceso denegado y termina el caso de uso.
2 – Exc.	Sin permisos: el sistema muestra mensaje de acceso denegado y termina el caso de uso.
5 – Exc.	Sin registros en el período: el sistema muestra el reporte con todos los totales en cero.

CU-03 – Exportar Reporte

Caso de Uso	Exportar Reporte	Identificador: CU-03
Actores	Administrador del Sistema (principal), Comité de Administración	
Tipo	Secundario	
Referencias	RF-29. Relacionados: CU-01, CU-02 (este CU es extendido por ambos).	
Precondición	Debe existir un reporte previamente generado en la sesión actual. El actor debe estar autenticado.	
Postcondición	El sistema descarga el archivo del reporte en el formato seleccionado al dispositivo del actor.	
Descripción	Permite exportar un reporte ya generado en el formato seleccionado. Es invocado como extensión de CU-01 y CU-02.	
Resumen	El actor selecciona Exportar sobre un reporte generado, elige el formato y el sistema produce y descarga el archivo.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Administrador	Selecciona la opción Exportar sobre el reporte generado en pantalla.
2	Sistema	Presenta los formatos de exportación disponibles: PDF, Excel (.xlsx), CSV.
3	Administrador	Selecciona el formato deseado y confirma la exportación.
4	Sistema	Genera el archivo del reporte en el formato seleccionado.
5	Sistema	Descarga el archivo al dispositivo del actor. Fin del caso de uso.

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
3a	El actor cancela la selección: el sistema cierra el diálogo y retorna al reporte en pantalla.
4 – Exc.	Error de procesamiento: el sistema muestra mensaje de error específico y ofrece reintentar.

4.1.1. CU-04 – Registrar Estado de Morosidad Automáticamente

Caso de Uso	Registrar Estado de Morosidad Automáticamente	Identificador: CU-04
Actores	Sistema — proceso automático (principal), Administrador del Sistema	
Tipo	Secundario	
Referencias	RF-10, RF-07, RF-11, RF-12, RF-14. Relacionados: CU-01.	
Precondición	Existen gastos comunes con fecha de vencimiento cumplida sin pago confirmado por el Sistema de Pagos Externo.	
Postcondición	Las unidades con deuda vencida quedan con estado de morosidad activo. El Servicio de Notificaciones es invocado. La operación queda en el historial de auditoría.	
Descripción	Proceso automático que se dispara al detectar gastos comunes vencidos sin pago confirmado. Registra morosidad y notifica al residente.	
Resumen	El sistema verifica vencimientos, identifica unidades sin pago confirmado, activa estado de morosidad y delega notificación al Servicio de Notificaciones externo.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Sistema	Ejecuta proceso de verificación de morosidad (disparado automáticamente por fecha de vencimiento).
2	Sistema	Consulta gastos comunes con fecha de vencimiento alcanzada en el período vigente.
3	Sistema	Verifica si existe pago confirmado por el Sistema de Pagos Externo para cada unidad en el período.
4	Sistema	Para cada unidad sin pago confirmado, registra el estado de morosidad como activo.
5	Sistema	Invoca el Servicio de Notificaciones para enviar alerta de morosidad al residente afectado.
6	Sistema	Registra la operación completa en el historial de auditoría. Fin del caso de uso.

CURSO ALTERNATIVO

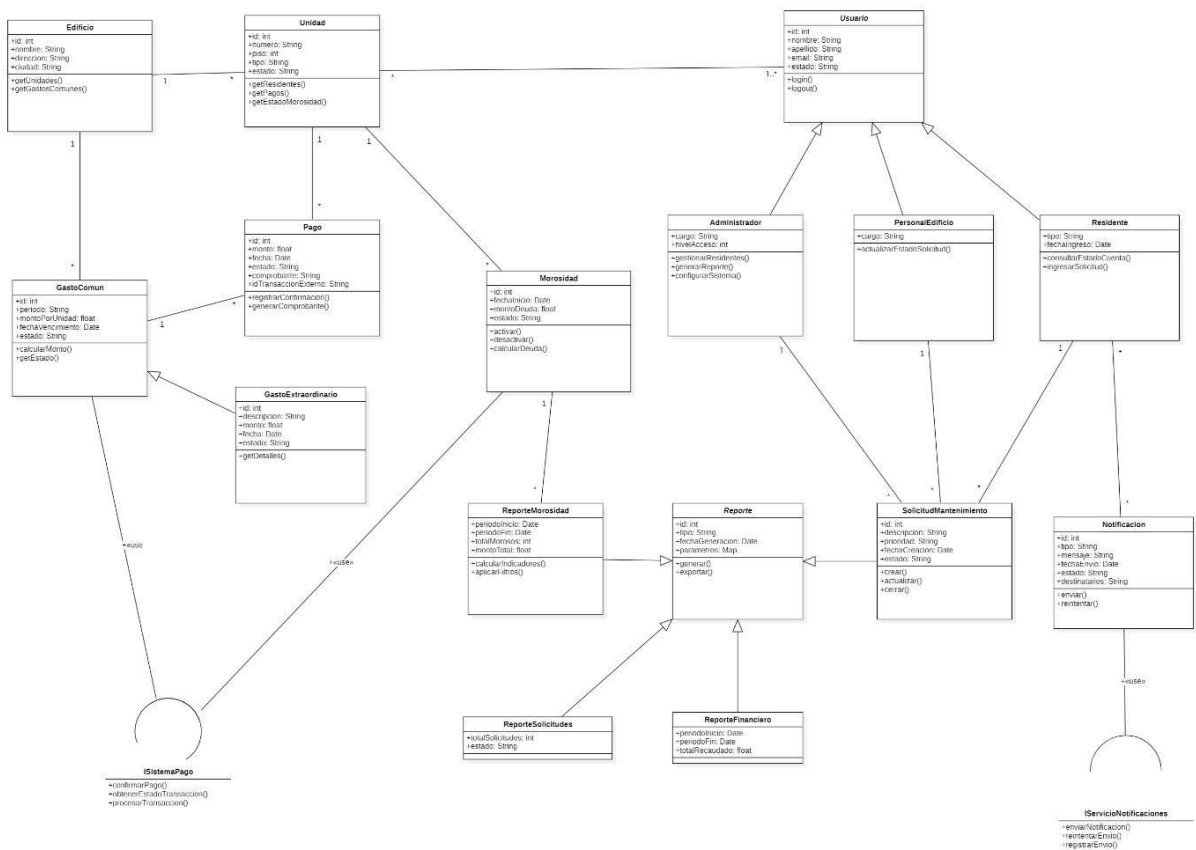
Nro.	Descripción de acciones alternas
3a	Pago confirmado posterior al vencimiento: el sistema no activa morosidad para esa unidad y continúa con la siguiente.
4a	Unidad ya morosa de período anterior: el sistema actualiza el monto acumulado sin crear registro duplicado.
5 – Exc.	Servicio de Notificaciones no disponible: el sistema registra el fallo y programa reintento (RNF-14). La morosidad se registra igualmente.
Post-activ.	Si se registra pago confirmado posterior para una unidad morosa, el sistema desactiva automáticamente el estado de morosidad.

22 VISTA LÓGICA

2.2.1. Propósito

La Vista Lógica describe la estructura estática del sistema desde el punto de vista del diseño orientado a objetos. Su propósito es identificar las clases principales del dominio, sus atributos, operaciones y las relaciones entre ellas. Para el Sistema El Mirador, esta vista organiza las entidades del dominio en torno a las tres capas de la arquitectura adoptada: clases de dominio (Capa de Datos), servicios (Capa de Lógica de Negocio) e interfaces hacia sistemas externos.

2.2.2. Diagrama de clases



2.2.3. Descripción diagrama de clases

El diagrama de clases representa el modelo de dominio del Sistema El Mirador, estructurado en torno a cuatro categorías conceptuales. La entidad principal es el Edificio, que incluye varias Unidades Habitacionales, cada una vinculada a uno o más Usuarios (Administrador, Residente o Personal). La jerarquía de Usuario agrupa los atributos de identificación y acceso, especializándose en los tres tipos de actores internos del sistema mediante un enfoque de herencia. El módulo financiero conecta las clases GastoComun, GastoExtraordinario, Pago y Morosidad. GastoExtraordinario es una

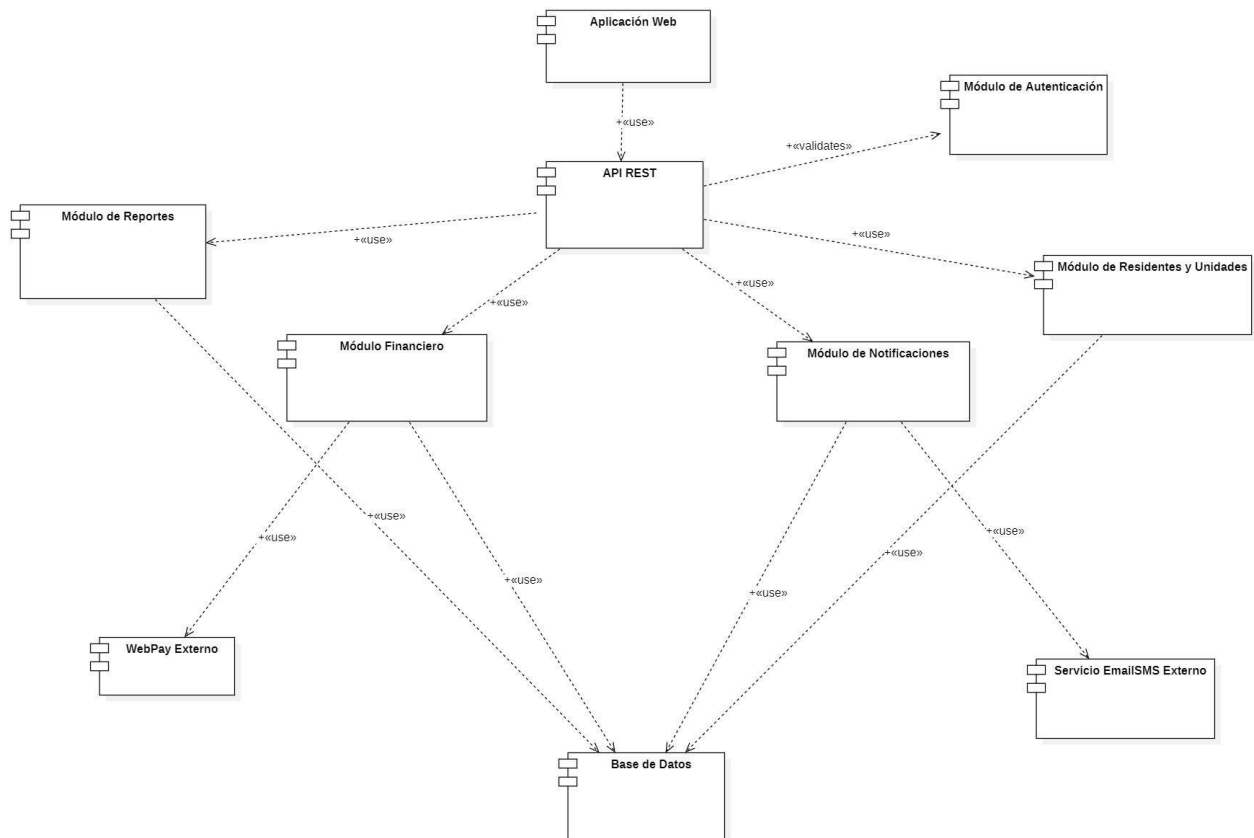
especialización de GastoComun, indicando esta relación de herencia. Los pagos son registrados únicamente a través de las confirmaciones del Sistema de Pagos Externo (ISistemaPago), y se relacionan directamente con la Unidad que los genera. La clase Morosidad se activa y desactiva de manera automática según el estado de los pagos de la unidad correspondiente. El módulo de reportes utiliza el patrón Template Method a través de la clase abstracta Reporte, de la cual derivan ReporteMorosidad, ReporteFinanciero y ReporteSolicitudes. La clase Notificacion se conecta con Residente, lo que implica el envío de alertas a los residentes de las unidades implicadas. La clase Pago tiene una asociación directa con Unidad (con multiplicidad $\ast \rightarrow 1$), asegurando el seguimiento del pago hacia la unidad habitacional que lo genera. Las interfaces ISistemaPago e IServicioNotificaciones encapsulan la conexión con sistemas externos, protegiendo el dominio de modificaciones en las plataformas de terceros (RNF-18).

33 VISTA DE IMPLEMENTACIÓN/DESARROLLO

3.3.1. Propósito

La Vista de Implementación describe la organización estática del software en términos de módulos, componentes y paquetes de código fuente. Su propósito es mostrar cómo el sistema está estructurado para ser desarrollado, desplegado y mantenido. Para el Sistema El Mirador, esta vista evidencia la separación en capas y la dependencia entre los componentes del backend, frontend y los sistemas externos.

3.3.2. Diagrama de componente

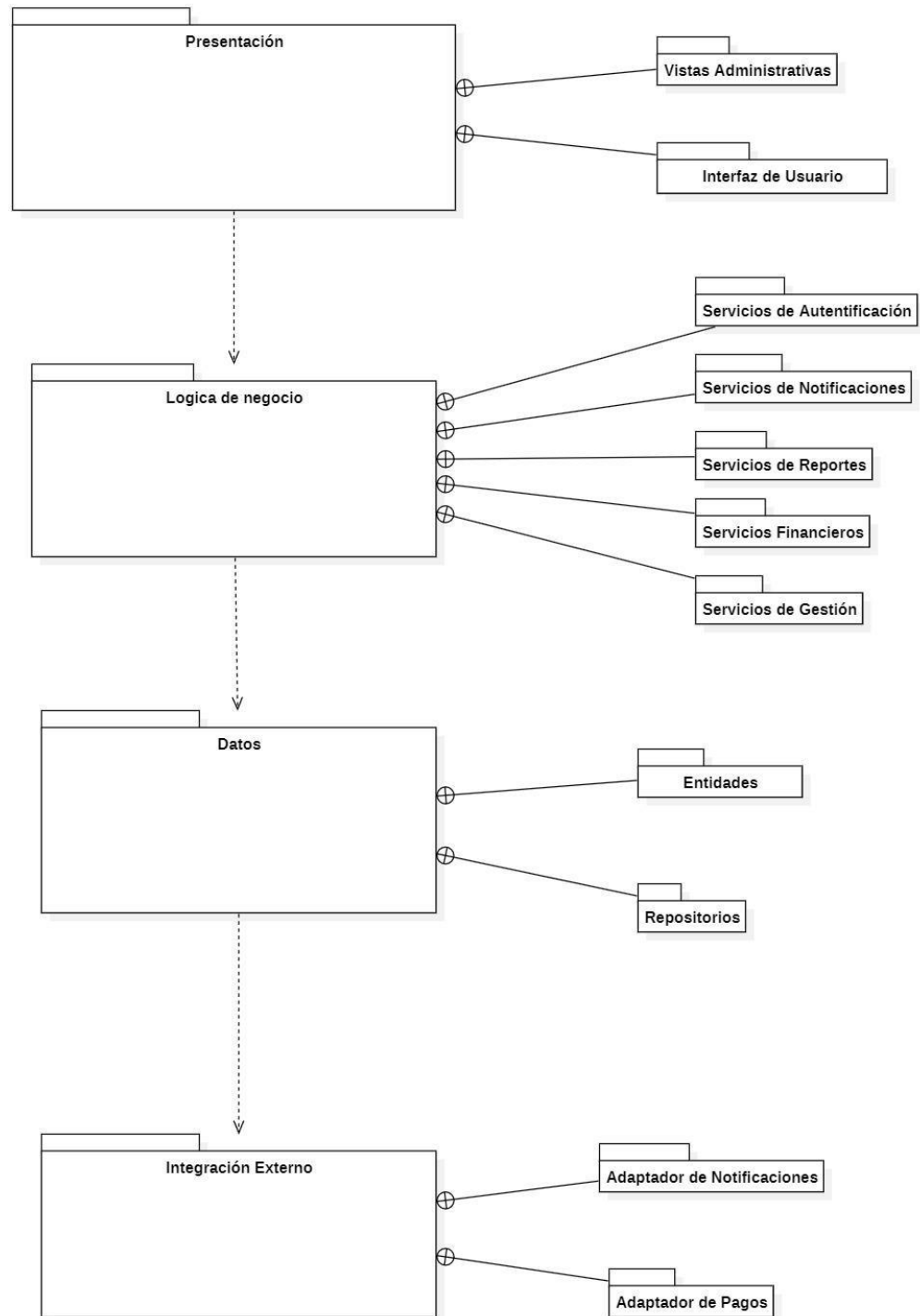


3.3.3. Descripción diagrama de componente

El diagrama de componentes refleja la implementación de la Arquitectura en Capas del sistema El Mirador. La Aplicación Web funciona como capa de presentación y se comunica únicamente con la API REST a través de HTTPS. La API REST coordina las operaciones entre los cinco módulos funcionales (Autenticación; Residentes y Unidades; Financiero; Notificaciones; Reportes). Cada módulo realiza su propio acceso a la base de datos de manera independiente. El Módulo Financiero delega el procesamiento de pagos a WebPay mediante su API REST, recibiendo solo el resultado o estado de la transacción. El Módulo de Notificaciones encarga el envío de mensajes al servicio externo de Email/SMS. Esta organización asegura que los componentes externos puedan sustituirse

sin afectar la lógica interna (RNF-18).

3.3.4. Diagrama de paquete



3.3.5. Descripción diagrama de paquete

El diagrama de paquetes muestra cómo se organiza lógicamente el código según la Arquitectura en Capas adoptada. Cada capa agrupa los paquetes correspondientes a su responsabilidad y mantiene dependencias en una sola dirección. En la Capa de Presentación se encuentran las vistas orientadas al usuario. La Capa de Lógica de Negocio centraliza los servicios funcionales y constituye el único punto de acceso a las Capas de Datos e Integración. La Capa de Datos agrupa los repositorios y los modelos de entidad. La Capa de Integración contiene los adaptadores para sistemas externos y aplica el patrón Fachada.

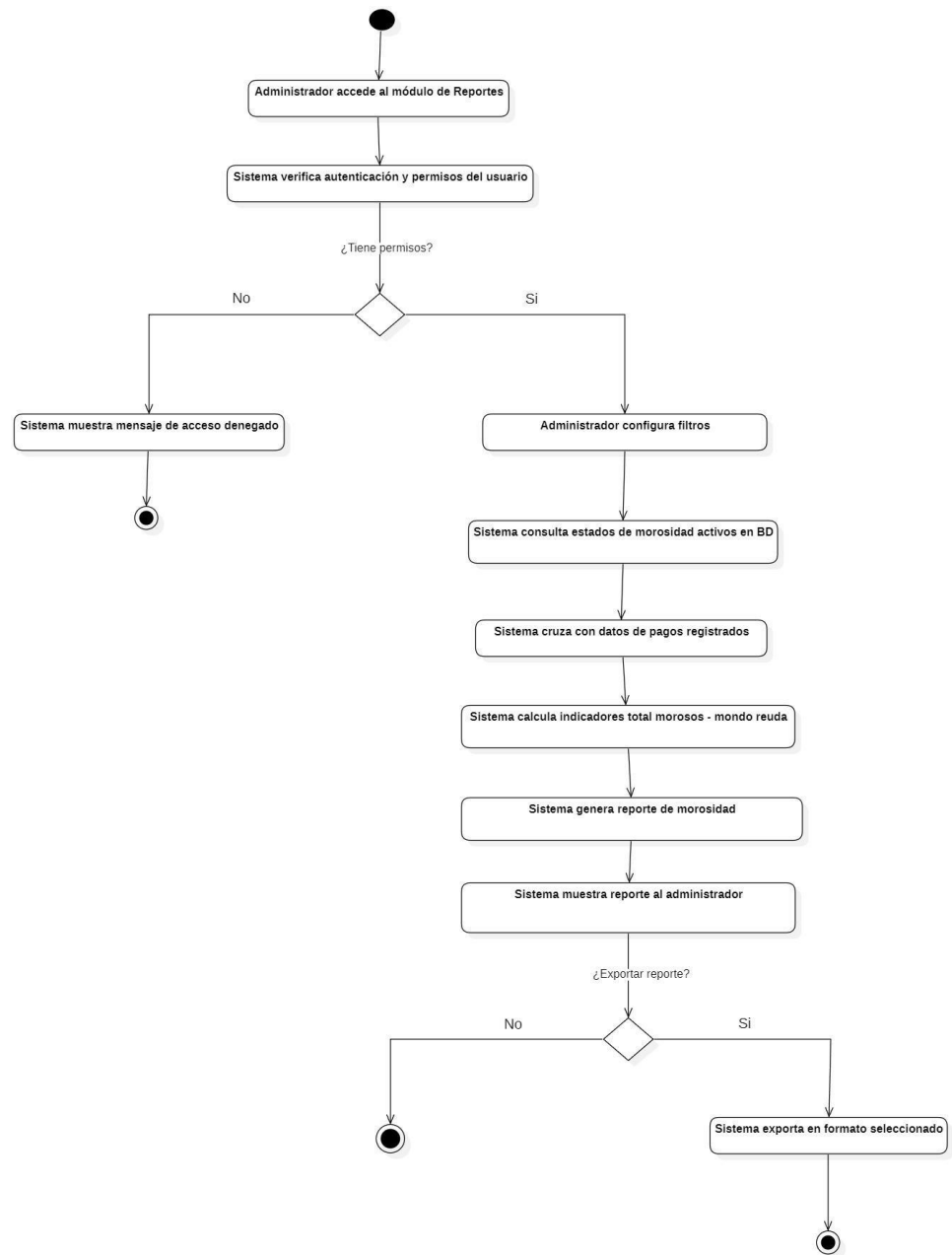
44 VISTA DE PROCESOS

4.4.1. Propósito

La Vista de Procesos describe el comportamiento dinámico del sistema, mostrando cómo las actividades y tareas se ejecutan en tiempo de ejecución, los flujos de control y las decisiones críticas. Para el Sistema El Mirador, esta vista se centra en el flujo del proceso de generación del Reporte de Morosidad (RF-24), por ser el caso de uso de

mayor convergencia funcional del sistema.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El diagrama de actividad muestra el flujo de ejecución para la generación del Reporte de Morosidad (RF-24). El proceso comienza cuando el Administrador ingresa al módulo de Reportes. Un primer punto de decisión comprueba la autenticación y los permisos; si el usuario no está autorizado, el proceso termina mostrando un mensaje de acceso

denegado. Con el acceso validado, el flujo avanza por cinco etapas: configuración de filtros, consulta del estado de morosidad, cruce con los registros de pagos confirmados, cálculo de indicadores y generación del reporte. Un segundo punto de decisión verifica si el usuario desea exportar el reporte; en caso afirmativo, el sistema crea el archivo correspondiente antes de finalizar. Este flujo deja en evidencia la dependencia de RF-24 respecto a RF-07, RF-10 y RF-29.

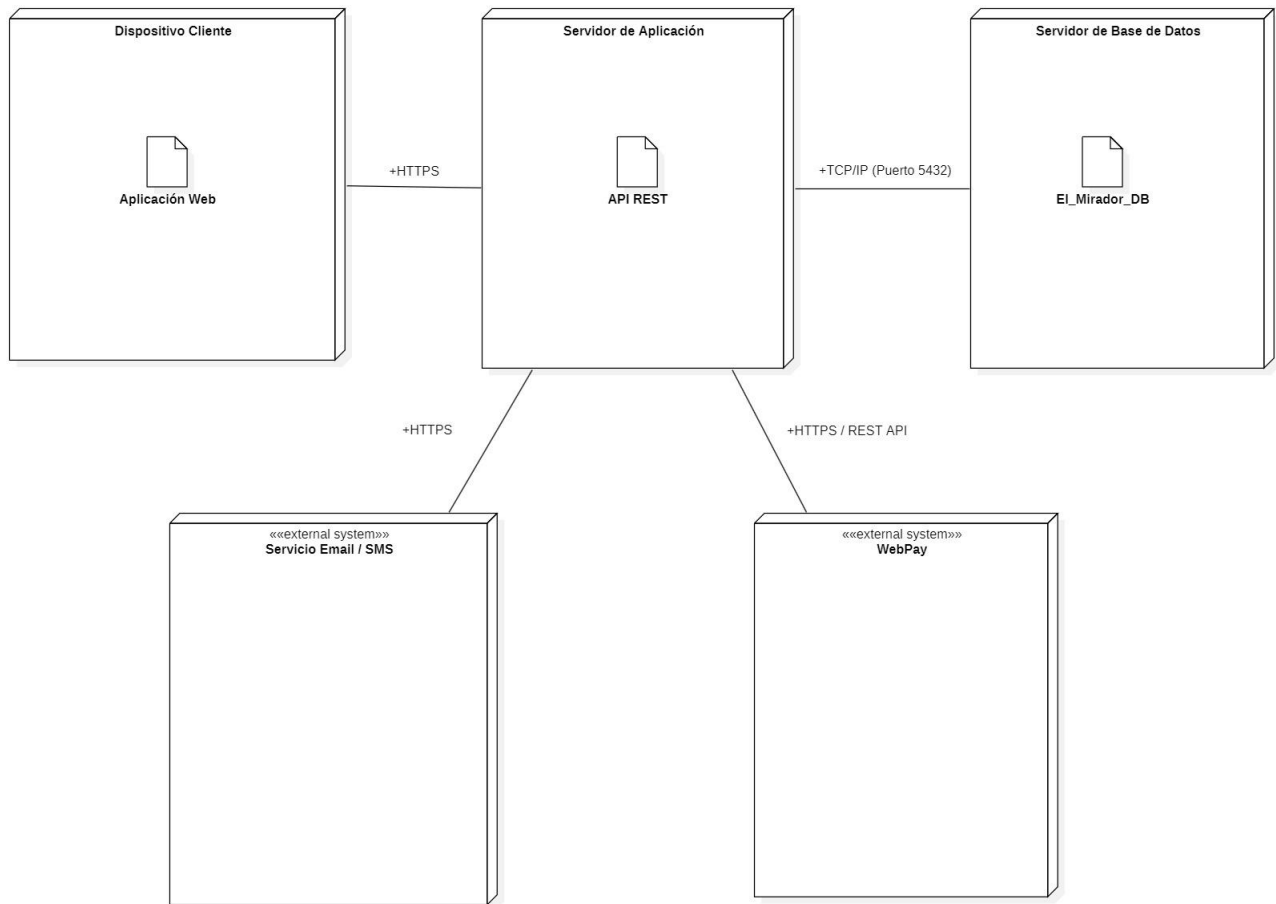
55 VISTA FÍSICA

5.5.1. Propósito

La Vista Física describe la distribución del sistema en la infraestructura de hardware y software de producción. Su propósito es mostrar los nodos sobre los que se ejecutan los componentes del sistema, los artefactos desplegados y los protocolos de comunicación entre ellos. Para el Sistema El Mirador, esta vista es especialmente relevante dado que

integra nodos internos con servicios externos (WebPay, Servicio de Notificaciones).

5.5.2. Diagrama de despliegue



5.5.3. Descripción diagrama de despliegue

El diagrama de despliegue muestra una arquitectura distribuida en tres capas físicas. El Dispositivo Cliente ejecuta la interfaz web o móvil y se conecta al Servidor de Aplicación únicamente mediante HTTPS (RNF-10, RNF-11). En el Servidor de Aplicación se alojan tanto el frontend como el backend (API REST), concentrando la lógica de negocio. El Servidor de Base de Datos está separado físicamente y se comunica con el servidor de aplicación vía TCP/IP (puerto 5432), lo que mejora la seguridad y permite escalar cada componente de forma independiente (RNF-06, RNF-11). Los nodos externos WebPay (estereotipo «external system») y Servicio Email/SMS (estereotipo «external system») son consumidos desde el Servidor de Aplicación mediante HTTPS, dejando claro que el sistema delega el procesamiento de pagos a infraestructura externa certificada (RNF-

18).

5. REQUISITOS DE CALIDAD

5.1. Propósito

Esta sección define los atributos de calidad que el Sistema de Administración El Mirador debe satisfacer, justificando su relevancia en el contexto del sistema y estableciendo los criterios concretos mediante los cuales se verificará su cumplimiento. Los atributos de calidad se derivan directamente de los requerimientos no funcionales (RNF) definidos en la sección 2.4 del presente documento, y se alinean con el estilo arquitectónico en capas adoptado en la sección 3

5.2. Atributos de calidad

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN
Usabilidad	La interfaz debe ser intuitiva y comprensible para administradores y residentes con distintos niveles de alfabetización digital (RNF-01).	El sistema gestiona flujos complejos (pagos, morosidad, reportes) que deben ser accesibles sin capacitación técnica avanzada.
Accesibilidad	La interfaz debe cumplir criterios WCAG 2.0 nivel AA (RNF-02).	El sistema es de uso comunitario y puede ser accedido por personas con distintas capacidades visuales o motrices.
Rendimiento	Las operaciones comunes de consulta deben responder en ≤ 3 segundos bajo carga normal (RNF-04).	Los módulos de reportes y consulta de morosidad manejan grandes volúmenes de datos y deben responder ágilmente.
Disponibilidad	El sistema debe estar disponible al menos el 99% del tiempo mensual, excluyendo mantenciones programadas (RNF-07).	La administración del edificio opera de forma continua. Indisponibilidad prolongada impacta directamente la gestión de pagos y morosidad.
Seguridad	Autenticación segura, control de acceso por roles y protección de datos personales y financieros (RNF-10, RNF-11).	El sistema maneja información sensible de residentes y transacciones económicas. El incumplimiento implica riesgos legales y de confianza.
Trazabilidad	Registro auditable de operaciones críticas: pagos, cambios de estado,	Esencial para la rendición de cuentas ante el comité de administración y para la

	notificaciones y accesos (RNF-12).	resolución de disputas entre residentes.
Mantenibilidad	Diseño modular que permita correcciones y mejoras sin afectar componentes no relacionados (RNF-15).	El sistema debe poder evolucionar (nuevos módulos, nuevas pasarelas de pago) sin necesidad de refactorizaciones completas.
Interoperabilidad	Integración con sistemas externos (WebPay, Servicio de Notificaciones) a través de APIs definidas (RNF-18).	El sistema delega el procesamiento de pagos y el envío de notificaciones a plataformas especializadas externas, lo que requiere interfaces robustas y documentadas.
Escalabilidad	El sistema debe adaptarse al crecimiento en usuarios, propiedades y transacciones sin degradar el rendimiento (RNF-06).	El sistema está diseñado para operar en múltiples edificios o comunidades en etapas futuras.
Confiabilidad	Consistencia de datos transaccionales y recuperación ante fallas (RNF-08, RNF-09).	La pérdida o inconsistencia de datos financieros o de morosidad tiene consecuencias directas en la gestión administrativa y en la confianza de los residentes.

5.3. Reglas y criterios de evaluación de calidad

A continuación se establecen los criterios medibles y los métodos de verificación mediante los cuales se evaluará el cumplimiento de cada atributo de calidad definido en la sección 5.1. Los criterios son específicos, medibles y vinculados a herramientas o técnicas de validación concretas.

ATRIBUTO DE CALIDAD	CRITERIO DE CUMPLIMIENTO	MÉTODO DE VERIFICACIÓN
Usabilidad	Prueba de usabilidad con usuarios reales (administrador y residente). Puntuación SUS (System Usability Scale) $\geq 75/100$.	Evaluación heurística de Nielsen (10 principios). Sesión de prueba con tareas representativas.
Accesibilidad	Cumplimiento WCAG 2.0 nivel AA en todas las pantallas principales.	Validador automático W3C (validator.w3.org). Inspección manual de contraste, navegación por teclado y etiquetas ARIA.
Rendimiento	Tiempo de respuesta de consultas ≤ 3 seg al 95% de	Prueba de carga con JMeter o herramienta equivalente.

	las peticiones bajo carga concurrente de 20 usuarios.	Medición de tiempos en módulo de reportes y consulta de morosidad.
Disponibilidad	Disponibilidad mensual $\geq 99\%$ (equivalente a ≤ 7.2 horas de indisponibilidad no programada al mes).	Monitoreo con herramienta de uptime (UptimeRobot o equivalente). Registro de incidentes y tiempos de caída.
Seguridad	0 vulnerabilidades críticas o altas en análisis de seguridad estático. Autenticación con token JWT y cifrado HTTPS en todas las rutas.	Análisis estático de código (OWASP ZAP o SonarQube). Revisión de configuración de roles y permisos por caso de uso.
Trazabilidad	100% de operaciones críticas (pagos, cambios de morosidad, notificaciones) registradas en log de auditoría con marca de tiempo, usuario y resultado.	Inspección de logs post-ejecución de pruebas funcionales. Verificación de integridad del registro de auditoría.
Mantenibilidad	Índice de mantenibilidad ≥ 70 (escala Visual Studio). Cobertura de pruebas unitarias $\geq 70\%$ en módulos críticos.	Análisis estático con SonarQube o CodeClimate. Revisión de métricas de acoplamiento y cohesión por módulo.
Interoperabilidad	Integración exitosa con WebPay en ambiente de sandbox: confirmación de pago recibida y registrada correctamente. Integración con Servicio de Notificaciones: envío y registro verificados.	Pruebas de integración en ambiente de pruebas con mocks de sistemas externos. Verificación de registro de transacciones y reintentos.
Escalabilidad	El sistema mantiene tiempo de respuesta ≤ 5 seg al aumentar la carga a 100 usuarios concurrentes.	Prueba de stress con JMeter. Monitoreo de uso de CPU y memoria en servidor de aplicación durante la prueba.
Confiabilidad	0 pérdidas de datos en pruebas de fallo simulado. Recuperación completa del sistema en ≤ 30 minutos desde backup.	Prueba de recuperación: simulación de fallo de BD y restauración desde backup. Verificación de integridad de datos post-restauración.

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

Las presente sección define los principios de diseño de software aplicados en la construcción del Sistema de Administración El Mirador. Estos principios guían las decisiones de arquitectura y desarrollo, garantizando que el sistema sea comprensible, mantenible, extensible y robusto a lo largo de su ciclo de vida. Su aplicación se refleja directamente en la organización de capas, el diagrama de clases y la separación de módulos funcionales del sistema.

6.2. Principios de diseño:

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones
Bajo Acoplamiento	Los módulos interactúan entre sí a través de interfaces definidas, minimizando las dependencias directas.	El Módulo Financiero no invoca directamente al Servicio de Notificaciones; lo hace a través de la interfaz IServicioNotificaciones, permitiendo reemplazar el proveedor sin modificar el módulo financiero.
Encapsulamiento	Los detalles de implementación de cada módulo están ocultos tras su interfaz pública.	La lógica de integración con WebPay está encapsulada en el Adaptador de Pagos (patrón Fachada). El resto del sistema solo conoce el resultado de la transacción, no el protocolo de comunicación.
Abstracción	Se modelan únicamente los aspectos relevantes del dominio del problema, omitiendo detalles no esenciales.	La clase abstracta Reporte define la interfaz común de todos los reportes del sistema. Las subclases concretas (ReporteMorosidad, ReporteFinanciero, ReporteSolicitudes) implementan el comportamiento específico de cada tipo.
Modularidad	El sistema está dividido en módulos funcionales independientes que pueden desarrollarse, probarse y desplegarse por separado.	Los módulos de Gestión de Residentes, Financiero, Notificaciones y Reportes son unidades independientes dentro de la capa de Lógica de Negocio, sin dependencias cruzadas entre ellos.
Separación de Responsabilidades	Las responsabilidades del sistema están distribuidas en capas con roles claramente distintos.	La Capa de Presentación gestiona la interfaz; la Capa de Lógica de Negocio aplica las reglas del dominio; la Capa de Datos gestiona la persistencia. Ninguna capa asume responsabilidades de otra.
Abierto/Cerrado	Los módulos están abiertos para extensión pero cerrados para modificación.	El sistema puede incorporar nuevos tipos de reporte heredando de la clase abstracta Reporte, sin modificar la lógica existente. Nuevas pasarelas de pago se integran implementando la interfaz ISistemaPago.
DRY (Don't Repeat Yourself)	La lógica que se repite se centraliza en un único lugar del sistema.	La verificación de autenticación y permisos está centralizada en el Módulo de Autenticación, invocado por todos los demás módulos. No existe lógica de control de acceso duplicada en cada módulo funcional.

7. PROTOTIPO

7.1. Propósito

Esta sección presenta el prototipo funcional del Sistema de Administración El Mirador, desarrollado en Figma. El prototipo materializa visualmente los flujos definidos en la arquitectura, centrándose en el requerimiento funcional RF-24 Reporte de Morosidad como flujo principal completamente navegable. Los módulos restantes del sistema se presentan como pantallas de contexto que permiten comprender la estructura general de la solución sin estar completamente desarrollados en esta fase.

7.2. Mockups (imágenes con una breve descripción)

Figura 1: Pantalla de acceso al sistema El Mirador. Presenta la interfaz de autenticación con campos de correo electrónico y contraseña, acceso protegido con autenticación de dos factores según RNF-10.

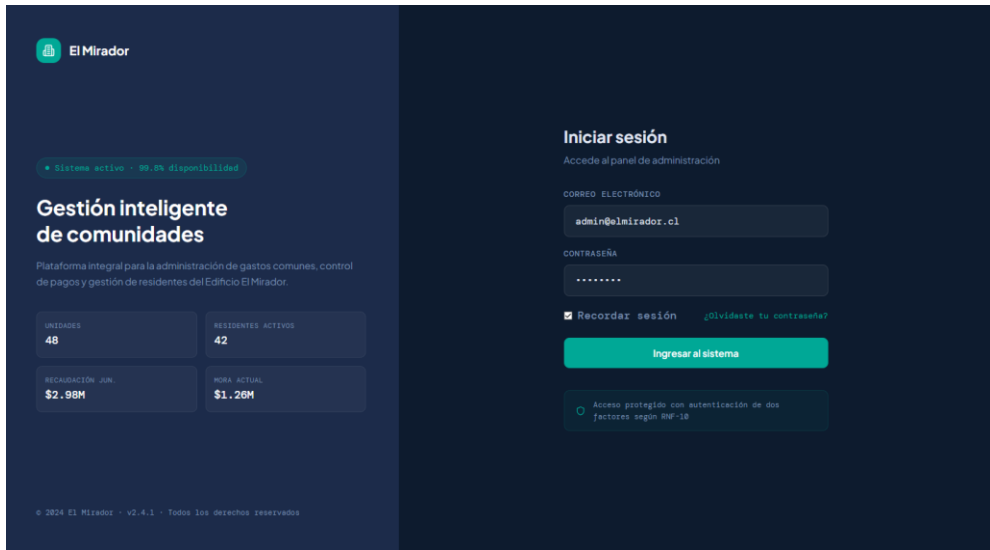


Figura 2: Dashboard principal de administración. Muestra indicadores clave: total de unidades, residentes activos, unidades morosas y mora total. Incluye gráfico de tendencia financiera y últimos pagos registrados

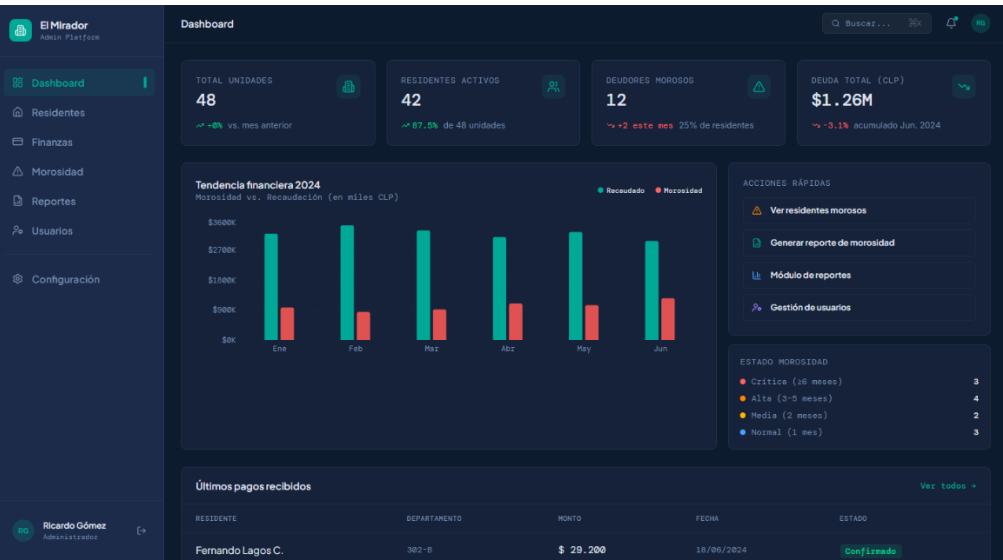


Figura 3: Gestión de residentes. Módulo en desarrollo, no figura en el flujo principal del RF asignado

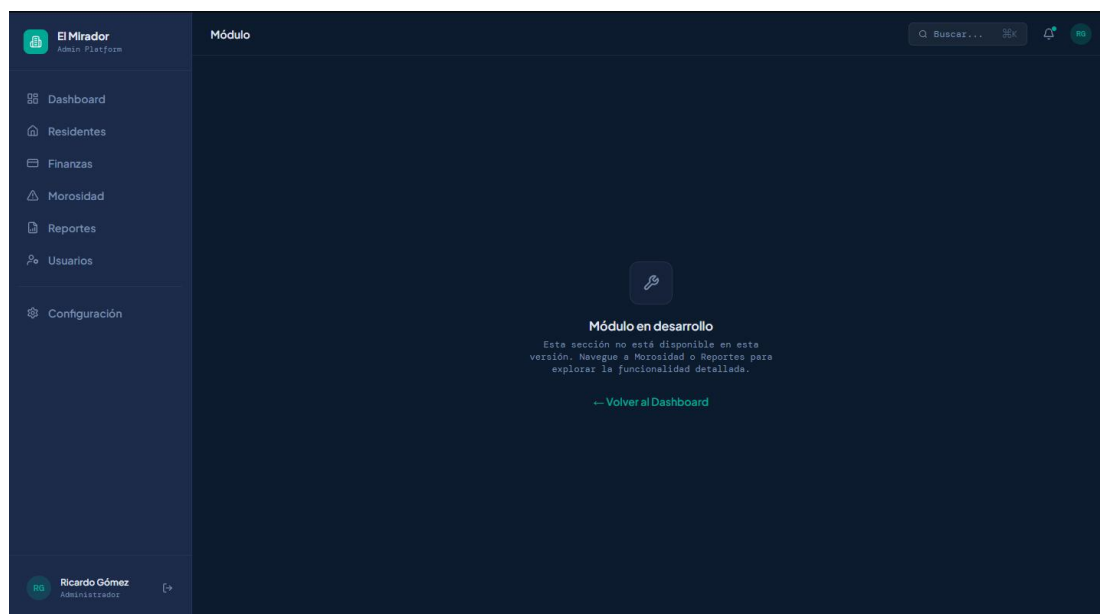


Figura 4: Gestión financiera de la comunidad. Módulo en desarrollo, no configura el flujo principal del RF asignado.

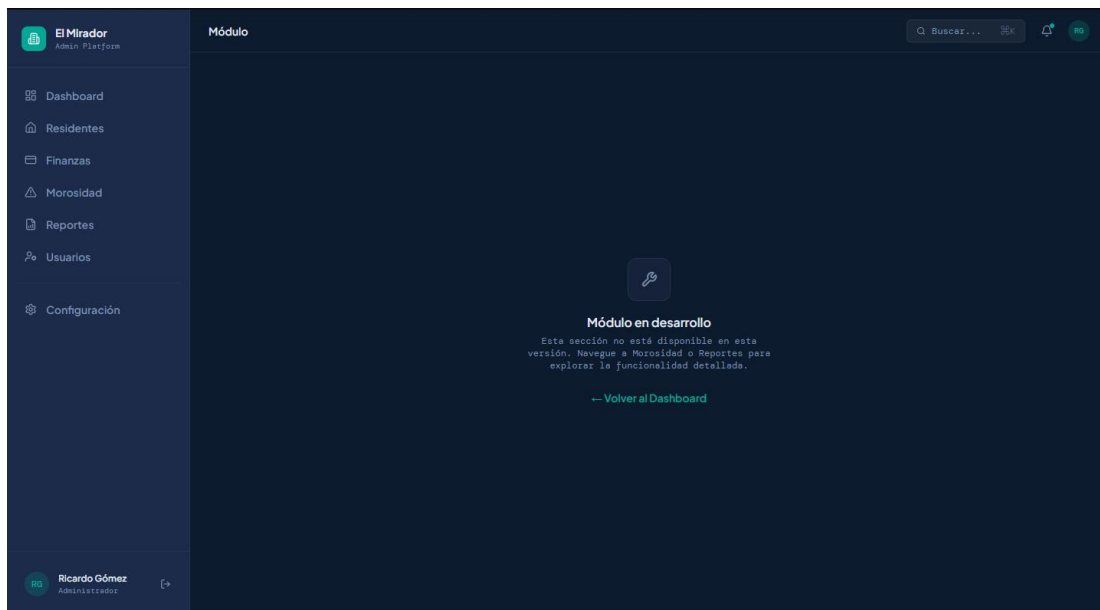


Figura 5: Control de residentes morosos. Lista completa con filtros por estado y tipo, monto adeudado, último pago y nivel de criticidad. Botón de acceso directo a generación de reporte.

El Mirador

Admin Platform

Dashboard

Residentes

Finanzas

Morosidad

Reportes

Usuarios

Configuración

RG

Ricardo Gómez

Administrador

Residentes morosos

Q

Buscar...

36K

🔔

RG

Residentes morosos

12 de 12 registros · Deuda total: \$ 1.200.000

Generar reporte

Filtros:

Estado: Todos

Tipo: Todos

Limpiar

RESIDENTE	DEPARTAMENTO	TIPO	MESES MORA	MONTO ADEUDADO	ÚLTIMO PAGO	NIVEL	
Carlos Mendoza R.	401-A	Propietario	3 meses	\$ 87.600	15/03/2024	Alta	Ver detalle
Patricia Soto V.	200-B	Arrendatario	1 mes	\$ 29.200	20/05/2024	Normal	Ver detalle
Jorge Villanueva C.	1102-A	Propietario	6 meses	\$ 175.200	01/12/2023	Crítica	Ver detalle
Ana María Fuentes	300-C	Propietario	2 meses	\$ 58.400	10/04/2024	Media	Ver detalle
Roberto Espinoza M.	707-A	Arrendatario	4 meses	\$ 116.800	28/02/2024	Alta	Ver detalle
Claudia Herrera O.	902-B	Propietario	1 mes	\$ 29.200	18/05/2024	Normal	Ver detalle
Diego Contreras P.	601-A	Propietario	8 meses	\$ 233.600	05/10/2023	Crítica	Ver detalle
Valeria Morales T.	1001-C	Arrendatario	2 meses	\$ 58.400	22/04/2024	Media	Ver detalle
Andrés Pérez F.	503-B	Propietario	5 meses	\$ 146.000	30/01/2024	Alta	Ver detalle
Francisca Núñez A.	802-A	Propietario	3 meses	\$ 87.600	08/03/2024	Alta	Ver detalle
Miguel Torres L.	1204-B	Arrendatario	7 meses	\$ 204.400	15/11/2023	Crítica	Ver detalle

Figura 6: Módulo de reportes. Vista general con todos los tipos de reporte disponibles. El Reporte de Morosidad aparece como funcionalidad activa.

El Mirador

Admin Platform

Dashboard

Residentes

Finanzas

Morosidad

Reportes

Usuarios

Configuración

RG

Ricardo Gómez

Administrador

Módulo de reportes

Q

Buscar...

36K

🔔

RG

Módulo de reportes

Selecciona el tipo de reporte a generar

Informe general

Informe general

Resumen integral de la gestión del edificio con parámetros configurables.

Próximamente disponible

Reporte financiero

Reporte financiero

Estado de ingresos, egresos y flujo de caja del periodo.

Próximamente disponible

Reporte de morosidad

Reporte de morosidad

ANÁLISIS DE DEUDORES MOROSOS CON FILTROS AVANZADOS Y EXPORTACIÓN.

Configurar filtros

Reporte de pagos

Reporte de pagos

Historial de pagos realizados con criterios configurables de búsqueda.

Próximamente disponible

Gastos comunes

Gastos comunes

Reporte de gastos comunes y extraordinarios del periodo.

Próximamente disponible

Estado de cuenta

Estado de cuenta

Estado de cuenta individual por residente o unidad habitacional.

Próximamente disponible

Reportes automáticos programados

Los reportes de morosidad se generan automáticamente el primer día de cada mes y se envían a los correos configurados en el sistema de notificaciones.

Configurar

Figura 6.1: Configuración de filtros del Reporte de Morosidad (RF-24). Permite definir período, piso, tipo de residente, estado de morosidad, monto mínimo y criterio de ordenamiento antes de generar el reporte.

El Mirador
Admin Platform

Dashboard

Residentes

Finanzas

Morosidad

Reportes

Usuarios

Configuración

Ricardo Gómez
Administrador

Reportes > Configuración del reporte

Reporte de Morosidad
Configure los filtros para generar el reporte

12 registros coinciden

PERÍODO

DESDE (AAAA-MM)2024-01HASTA (AAAA-MM)2024-06

UNIDAD HABITACIONAL

PISOTodosTIPO DE RESIDENTETodos

MOROSIDAD

ESTADO DE MOROSIDADTodosMONTO MÍNIMO ADEUDADO (CLP)0

ORDENAMIENTO

ORDENAR PORmonto-desc

Restablecer filtrosGenerar reporte (12)

Figura 6.2: Resultado del Reporte de Morosidad generado. Muestra tabla detallada con residente, departamento, tipo, meses de mora, monto adeudado, último pago y nivel de criticidad.

El Mirador
Admin Platform

Dashboard

Residentes

Finanzas

Morosidad

Reportes

Usuarios

Configuración

Ricardo Gómez
Administrador

Reportes > Configuración del reporte > Resultados del reporte

Resultados del reporte
Período 2024-01 - 2024-06 · Generado el 18/06/2024 10:35

Exportar reporte

TOTAL DEUDORES
12
residentes morosos

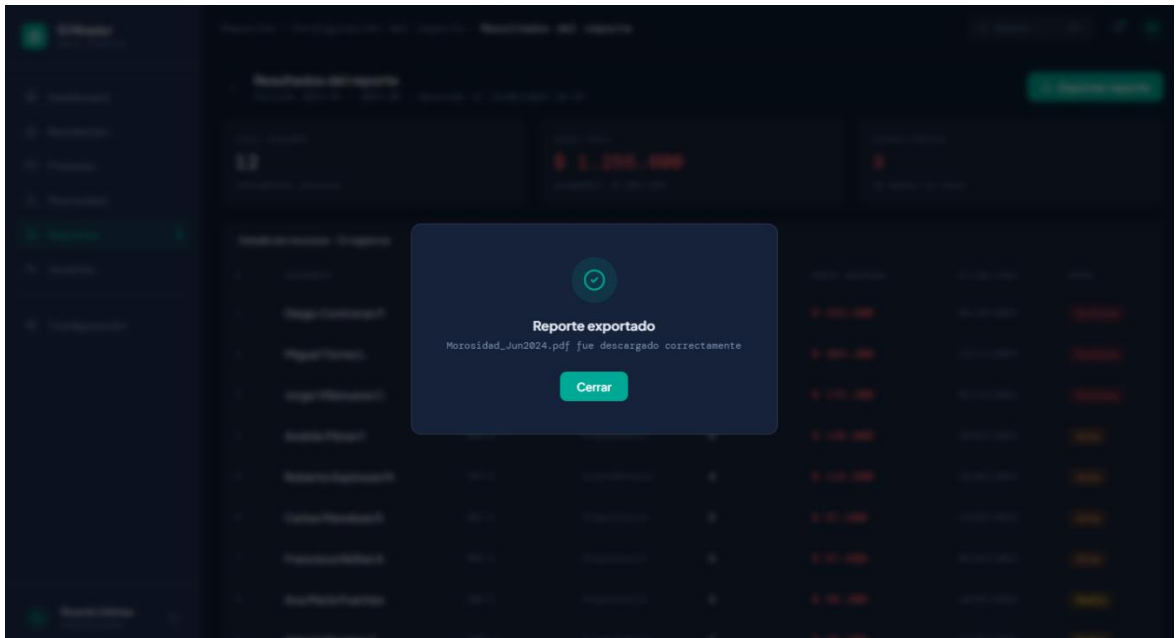
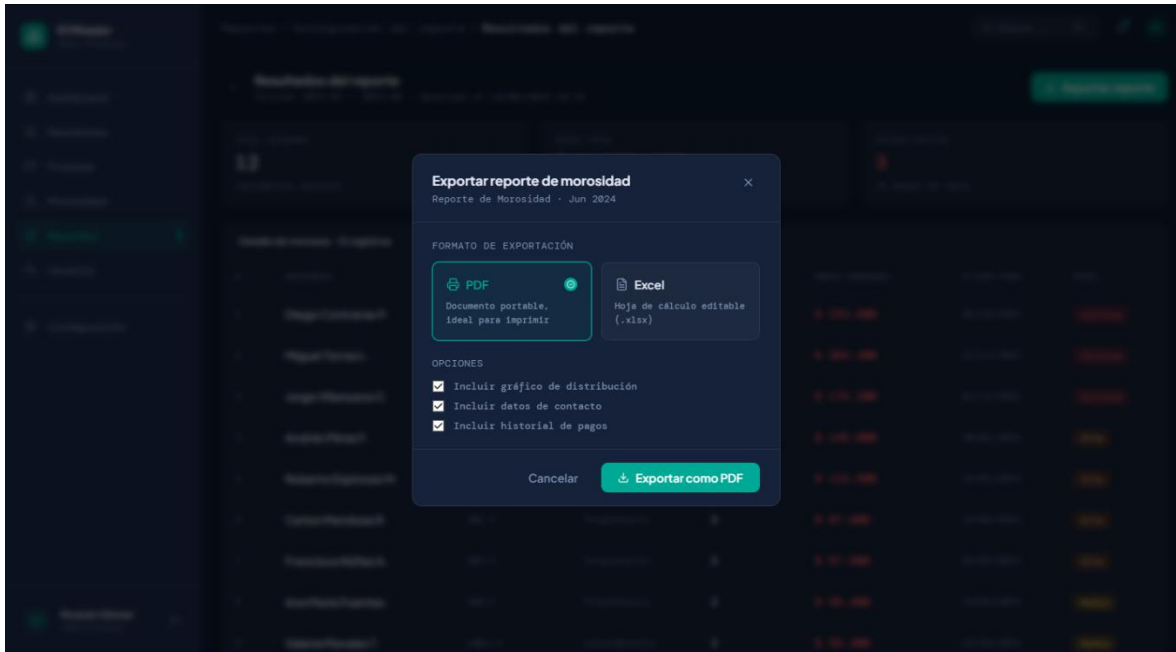
DEUDA TOTAL
\$ 1.255.600
promedio: \$ 104.633

ESTADO CRÍTICO
3
26 meses en mora

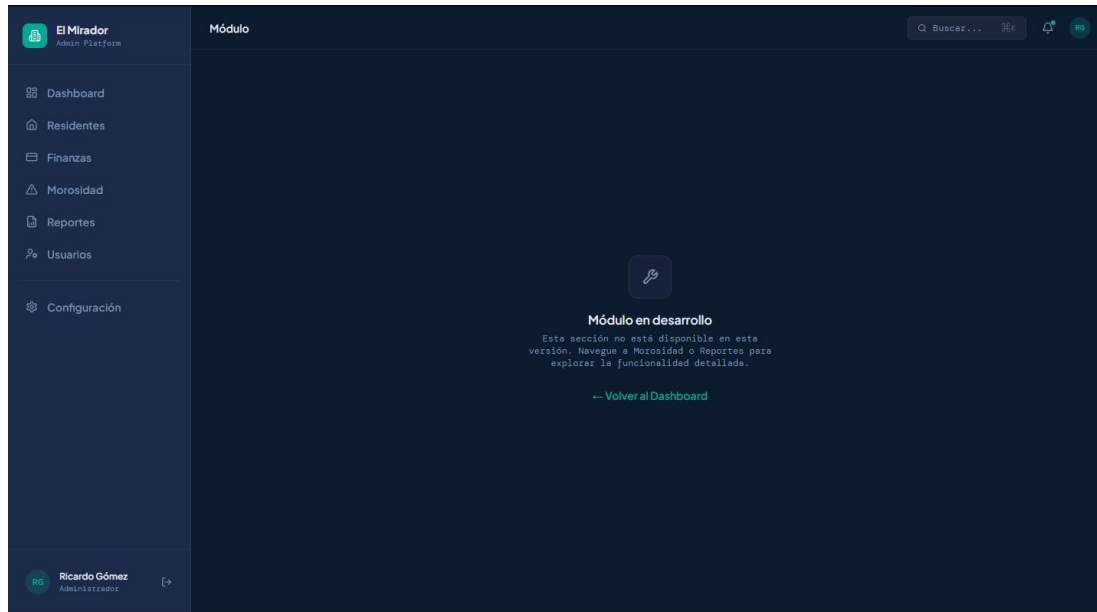
Detalle de morosos · 12 registros

#	RESIDENTE	DEPARTAMENTO	TIPO	MESES MORA	MONTO ADEUDADO	ÚLTIMO PAGO	NIVEL
1	Diego Contreras P.	601-A	Propietario	8	\$ 233.600	05/10/2023	Crítica
2	Miguel Torres L.	1204-B	Arrendatario	7	\$ 204.400	15/11/2023	Crítica
3	Jorge Villanueva C.	1102-A	Propietario	6	\$ 175.200	01/12/2023	Crítica
4	Andrés Pérez F.	503-B	Propietario	5	\$ 146.000	30/01/2024	Alta
5	Roberto Espinoza M.	707-A	Arrendatario	4	\$ 116.800	28/02/2024	Alta
6	Carlos Mendoza R.	401-A	Propietario	3	\$ 87.600	15/03/2024	Alta
7	Francisca Núñez A.	802-A	Propietario	3	\$ 87.600	08/03/2024	Alta
8	Ana María Fuentes	308-C	Propietario	2	\$ 58.400	10/04/2024	Media
9	Valeria Morales T.	1001-C	Arrendatario	2	\$ 58.400	22/04/2024	Media

6.3: Exportación del reporte. Modal de selección de formato (PDF o Excel) con opciones adicionales y pantalla de confirmación de descarga exitosa.



7: Administración de usuarios del sistema. Tabla con nombre, correo, rol, estado y último acceso. Permite agregar y editar usuarios según RNF-10.



7.3. Justificar herramientas de prototipado

Para la creación y desarrollo del prototipo del Sistema de Administración El Mirador se utilizó Figma como herramienta principal de prototipado. La elección se justifica por las siguientes razones: Figma es una herramienta colaborativa basada en navegador que no requiere instalación local, lo que facilita el acceso y la compartición del prototipo mediante enlace directo. Permite construir prototipos interactivos con navegación entre pantallas, simulando el comportamiento funcional del sistema sin necesidad de código o si bien utilizar su herramienta de componentes IA (Figma Make) para maximizar la agilidad en creación de proyectos con modificaciones a través de prompts, alineándose con los principios de diseño definidos en la sección 6. Figma es ampliamente adoptada en la industria del desarrollo de software para la fase de prototipado y validación de interfaces y finalmente, el prototipo generado se puede compartir mediante un enlace público al publicarlo, cumpliendo con los requisitos de entrega establecidos en la evaluación.

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

Esta sección presenta la evaluación de calidad del prototipo del Sistema de Administración El Mirador mediante la aplicación de las 10 heurísticas de usabilidad de Jakob Nielsen. El objetivo es identificar problemas de usabilidad en las interfaces diseñadas, clasificarlos por severidad y proponer mejoras concretas. La evaluación se centra en el flujo principal del requerimiento RF-24 Reporte de Morosidad y las pantallas de contexto que lo rodean.

8.2. Lista de verificación

La lista de verificación heurística se encuentra registrada en la planilla adjunta 'Evaluacion_Heuristica_Nielsen_Bastian.xlsx', la cual constituye el instrumento oficial de esta evaluación. Dicha planilla contiene el registro de problemas identificados por heurística, su ubicación en el prototipo, severidad asignada según la escala de Nielsen (0-4), evidencia y recomendación de mejora.

8.3. Análisis y métricas de resultados

Los resultados de la evaluación heurística aplicada al prototipo se encuentran consolidados en la hoja "Resumen" de la planilla adjunta "Evaluacion_Heuristica_Nielsen_Bastian.xlsx". El análisis considera la distribución de problemas por heurística y nivel de severidad, permitiendo priorizar las correcciones antes del lanzamiento del sistema. Los problemas de severidad 3 y 4 requieren corrección obligatoria; los de severidad 1 y 2 se abordan en iteraciones posteriores del prototipo.

9. CONTROL DE VERSIONES

9.1. Propósito

Esta sección documenta el sistema de control de versiones aplicado al desarrollo del software del Sistema de Administración El Mirador. El control de versiones permite registrar la evolución del diseño del prototipo, trazando cada iteración desde los wireframes iniciales hasta la versión funcional entregable, garantizando trazabilidad y facilitando la mejora continua del prototipo.

9.2. Control de versión utilizado

Se adoptó un sistema de versionamiento semántico (MAJOR.MINOR.PATCH) combinado con el historial nativo de versiones de Figma. El versionamiento semántico fue elegido por sobre el secuencial y el basado en fechas porque permite comunicar con claridad el tipo de cambio realizado: MAJOR indica rediseño estructural de pantallas, MINOR indica incorporación de nuevas pantallas o flujos, y PATCH indica correcciones visuales o ajustes menores. Esta elección es coherente con las buenas prácticas de la industria del desarrollo de software y permite al equipo evaluador identificar el alcance de cada iteración sin revisar el detalle de cada cambio. El historial de versiones de Figma complementa este esquema registrando automáticamente cada modificación mientras que GitHub actúa como repositorio central de todos los artefactos del proyecto.

9.3. Justificar herramientas de versionamiento

Se utilizaron dos herramientas complementarias para el control de versiones del proyecto. Figma como herramienta principal de prototipado incorpora un sistema de versionamiento automático que registra cada cambio realizado sobre el diseño con marca de tiempo y descripción del autor. Esto permite recuperar cualquier estado anterior del prototipo y comparar versiones de forma visual. GitHub fue utilizado como repositorio central del proyecto para almacenar y versionar todos los artefactos documentales generados durante las tres evaluaciones parciales del semestre. Aunque el proyecto no contiene código fuente, GitHub es la plataforma estándar de la industria para el control de versiones de proyectos de software, y su uso en este contexto demuestra capacidad de aplicar buenas prácticas de gestión de proyectos digitales. El repositorio se organiza en tres carpetas principales correspondientes a cada informe parcial entregado.

7. CONCLUSIONES

El desarrollo de este proyecto permitió aplicar de forma integrada los principios de la Ingeniería de Software, abarcando el ciclo completo desde el levantamiento de requerimientos hasta la creación de un prototipo funcional con criterios de calidad establecidos. Durante las tres evaluaciones parciales se diseñó y documentó la arquitectura del Sistema de Administración El Mirador siguiendo el Modelo 4+1; se especificaron 30 requerimientos funcionales y 18 no funcionales estandarizados; se construyó un prototipo interactivo focalizado en el RF-24 (Reporte de Morosidad) empleando Figma y Figma Make; y se evaluó la calidad mediante la norma ISO 25010 y la inspección heurística de Nielsen, lo que permitió identificar 15 problemas de usabilidad junto con recomendaciones de mejora. El empleo de herramientas profesionales (StarUML para el modelado UML, Figma para el prototipado y GitHub para el control de versiones) evidencia la aplicación práctica de estándares de la industria del software. La combinación de versionamiento semántico y el historial nativo de Figma aseguró trazabilidad completa de la evolución del diseño. Como aprendizajes principales se subrayan: la necesidad de definir con precisión los requerimientos antes de iniciar el diseño; la ventaja de enfocar el prototipo en el requerimiento que concentra mayor funcionalidad del sistema; y la importancia de documentar cada decisión arquitectónica para facilitar el mantenimiento y la escalabilidad futura.

8. BIBLIOGRAFÍA

Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.

ISO/IEC 25010:2011. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models. International Organization for Standardization.

Figma Inc. (2024). Figma Design Tool — Documentación oficial. Recuperado de <https://help.figma.com>

9. ANEXOS

9.1. Carta Gantt